

EyeRobot Functional Analysis

Pedro Conde Salvarelli, Gwenaëlle Queloz, Nathan Kack Kack

March 2026

Contents

1	Functional Analysis	2
1.1	System Requirements	2
1.2	System Functional Architecture	3
1.3	Functional Decomposition	3
1.3.1	Button Interaction	3
1.3.2	Localization & Obstacle Detection	5
1.3.3	Navigation & Planning	5
1.3.4	Block recovery	5
1.3.5	Power System	6
1.3.6	Computation Platform	7
1.3.7	Software Architecture	7
1.4	Subsystem Design Exploration	7
1.4.1	Wheel Solutions	8
1.4.2	Button Interaction Solutions	8
1.4.3	High-level localization solutions	9
1.4.4	Obstacle avoidance and block detection, collection and delivery solutions	11
1.4.5	Power system solutions	14
1.4.6	On Board Computer solutions	14
1.4.7	Localization solutions	15
1.4.8	Navigation & Path Planning solutions	15
1.4.9	Middleware solutions	17
1.5	Mission Strategy	17
2	Project Timeline	19
2.1	Up to date BOM	21

1 Functional Analysis

The functional analysis translates the competition brief into engineering functions that can be allocated to mechanical, electronic, and software subsystems. The following solutions are sorted by order of preference, with our preferred solutions on top of each table.

1.1 System Requirements

Table 1 exposes the competition objectives and constraints as explicit system requirements.

ID	Requirement	Interpretation
REQ-01	The robot shall operate autonomously throughout a competition run without remote control input.	The competition rules require autonomous behavior, so the control architecture must execute the mission without operator intervention after the start signal.
REQ-02	The robot shall rely exclusively on onboard electrical power during the mission.	The platform must be battery powered and remain fully mobile during the run.
REQ-03	The robot shall perform all mission-critical computation onboard and shall not depend on remote processing or remote decision support.	The competition forbids remote computation, which constrains both the hardware platform and the software architecture.
REQ-04	The robot shall detect Duplo blocks and obstacles in the arena environment.	Perception is necessary both for block collection and for safe navigation through the arena.
REQ-05	The robot shall estimate its pose with respect to the arena sufficiently to reach blocks, the door button, and the collection corner.	Localization is required for goal-directed navigation, button interaction, and reliable delivery.
REQ-06	The robot shall navigate on the accessible competition surfaces, including the standard floor and the carpeted zone.	The arena contains multiple surface conditions that affect traction, motion control, and route planning.
REQ-07	The robot shall actuate the button that opens the door to Zone 3 whenever that zone is included in the mission strategy.	Access to the door-protected zone requires a dedicated interaction function beyond free-space navigation.
REQ-08	The robot shall traverse the ramp to Zone 4 whenever that zone is included in the mission strategy	The torque of the motors and the weight distribution need to be taken into account on the mechanical and electrical design process.
REQ-09	The robot shall collect blocks from the floor, retain them during transport, and store them until delivery.	The scoring objective depends on reliable block acquisition and payload release.
REQ-10	The robot shall detect when its storage capacity is full and trigger a return-to-corner delivery sequence.	Storage fullness is a decision point for delivery and mission resumption.
REQ-11	The robot shall release collected blocks at the collection corner in a controlled and repeatable manner.	The best scoring is achieved by depositing the payload reliably at the designated location.
REQ-12	The robot shall prioritize feasible targets and zone transitions in order to maximize the score achieved during the limited competition time.	The mission objective is not only to collect blocks, but to do so with an effective strategy under time and accessibility constraints.
REQ-13	The robot shall preserve the physical integrity of the blocks during collection, transport, storage, and delivery.	The competition rules require the blocks to remain intact, which constrains the design of the intake and release mechanisms.
REQ-14	The robot shall comply with competition envelope and operational constraints, including the prohibition of flying solutions.	Mechanical design choices must remain within the allowed robot class and arena constraints.

Table 1: System requirements derived from the competition objectives and design constraints.

1.2 System Functional Architecture

The robot is organized as a set of interacting functional subsystems. Perception and localization maintain an internal representation of the arena, mission planning selects profitable actions, motion control executes the required trajectories, and the collection, storage, and delivery subsystems handle the physical transfer of blocks from the floor to the collection corner. Supporting compute and power subsystems enable this closed-loop autonomy.

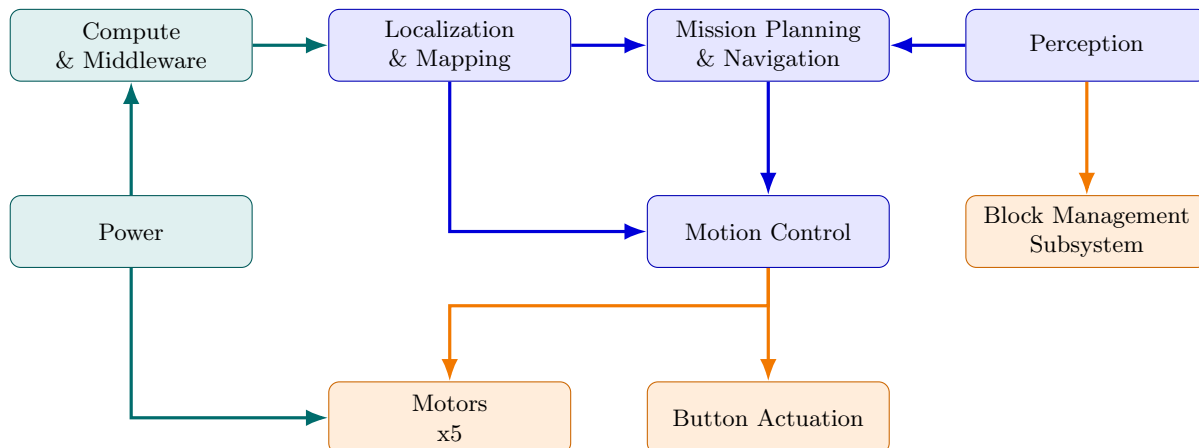


Figure 1: High-level functional architecture of the autonomous robot. Green: power/compute / Blue: software / Orange: actuators

1.3 Functional Decomposition

The functional decomposition expands the robot architecture into detailed mission-oriented subfunctions. The following subsections organize this decomposition by parent function and describe the specific operations that must be performed to achieve autonomous block collection, transport, and delivery.

1.3.1 Button Interaction

Subfunction	Description
Detect button location	Determine the position of the door-opening button with respect to the robot and the arena reference frame.
Align with button	Position and orient the robot so that the contact surface reaches the button with sufficient precision.
Apply contact force	Exert the force required to actuate the button and then clear the doorway for entry into Zone 3.

Table 3: Functional decomposition of the button-interaction subsystem.

Functional block	Main role	Inputs	Outputs	Interactions with other blocks
Perception	Detect blocks, obstacles, and relevant environmental features.	Camera, depth, LIDAR, or auxiliary range measurements.	Block detections, obstacle observations, target candidates.	Provides environment data to localization, navigation, and collection.
Localization and mapping	Estimate the robot pose and maintain the arena reference frame.	Encoder data, IMU data, range observations, visual cues.	Pose estimate, map updates, transforms to targets and delivery point.	Feeds navigation, motion control, and button interaction.
Mission planning and navigation	Select target blocks and compute feasible motion objectives.	Pose estimate, map, detected blocks, score priorities, storage state.	Target order, route requests, return-to-corner decisions.	Commands motion control and coordinates collection and delivery.
Motion control	Convert navigation objectives into drivetrain actuation.	Planned path, pose error, drivetrain state.	Wheel commands and chassis motion.	Executes navigation while enabling collection and button interaction maneuvers.
Environment interaction	Actuate the door-opening button when required.	Button target pose, robot pose, motion alignment state.	Contact action and access to Zone 3.	Depends on localization and motion control; updates mission planning once the zone is opened.
Collection subsystem	Acquire blocks from the floor and transfer them into storage.	Relative block pose, chassis motion, actuator commands.	Acquired blocks and intake status.	Works with perception for alignment and with storage management for payload handling.
Storage management	Retain collected blocks and monitor usable payload capacity.	Intake events, occupancy estimate, mechanical containment state.	Storage-full flag and payload state.	Informs mission planning and triggers delivery when necessary.
Delivery subsystem	Release blocks at the collection corner.	Corner pose, storage state, delivery command.	Deposited payload and post-delivery status.	Receives commands from mission planning and clears storage for the next cycle.
Compute and middleware	Execute the autonomy software stack and manage data exchange.	Sensor streams, mission state, control requests.	Processed data, synchronized transforms, subsystem messages.	Integrates all software functions on the onboard processor.
Power subsystem	Distribute battery energy and regulated rails to actuators and electronics.	Battery voltage and current.	Motor supply and regulated power outputs.	Supports sensing, computation, and actuation across the entire platform.

Table 2: High-level functional architecture of the autonomous robot.

1.3.2 Localization & Obstacle Detection

Subfunction	Description
Estimate robot pose	Estimate the robot position and heading with respect to the arena during navigation, collection, and delivery.
Fuse sensor measurements	Combine proprioceptive and exteroceptive measurements to reduce drift and improve pose consistency over the mission.
Maintain map reference	Preserve a consistent reference between the robot pose, the arena geometry, and mission targets such as blocks and the collection corner.

Table 4: Functional decomposition of the localization subsystem.

Subfunction	Description
Acquire environment data	Sense nearby surfaces and objects required for collision avoidance and route validation.
Detect obstacles	Distinguish occupied space from free space in the robot vicinity while the mission is in progress.
Estimate obstacle distance	Quantify the separation from walls and obstacles so that navigation can maintain safe clearance.

Table 5: Functional decomposition of the obstacle-detection subsystem.

1.3.3 Navigation & Planning

Subfunction	Description
Select next target	Choose the next block, zone, or return action in order to maximize score under time and accessibility constraints.
Compute route	Generate a feasible path from the current pose to the selected target or to the collection corner.
Update mission strategy	Revise the navigation objective when storage becomes full, access conditions change, or new blocks are detected.

Table 6: Functional decomposition of the navigation and planning subsystem.

1.3.4 Block recovery

Subfunction	Description
Acquire visual data	Capture the sensing information required to identify blocks under the lighting conditions of the competition arena.
Detect blocks	Identify candidate blocks in the observed scene and distinguish them from obstacles and background features.
Estimate block pose	Estimate the relative position of each detected block so that the robot can plan an approach for collection.
Control wheel actuation	Convert navigation commands into wheel-level actuation so that the robot can track planned trajectories and execute local maneuvers.

Table 7: Functional decomposition of the block-detection subsystem.

Subfunction	Description
Approach target block	Execute the final alignment needed to place the intake in a suitable capture configuration relative to the target block.
Intake block	Transfer the block from the arena floor into the robot through the selected collection mechanism.
Transfer block to storage	Guide the collected block from the intake region into the internal storage volume without losing or damaging it.

Table 8: Functional decomposition of the block-collection subsystem.

Subfunction	Description
Store collected blocks	Retain collected blocks inside the robot during navigation and before the delivery sequence.
Detect storage capacity	Determine when the storage volume is sufficiently full to trigger a return-to-corner delivery action.
Prevent block jams	Limit internal block interference so that intake and subsequent delivery remain reliable over repeated cycles.

Table 9: Functional decomposition of the block-storage subsystem.

Subfunction	Description
Transport payload to release point	Bring the stored blocks to the collection corner without premature payload loss.
Release blocks	Discharge the payload at the collection corner in a controlled manner.
Ensure reliable delivery	Confirm that the storage area has been emptied sufficiently before the robot resumes collection.

Table 10: Functional decomposition of the block-delivery subsystem.

1.3.5 Power System

Subfunction	Description
Supply traction power	Deliver current to the drive motors and mission actuators throughout the competition run.
Regulate electronics voltage	Provide voltage levels compatible with sensors, controllers, and onboard computing hardware.
Distribute electrical power	Route power to all subsystems while maintaining stable operation under varying load conditions.

Table 11: Functional decomposition of the power subsystem.

1.3.6 Computation Platform

Subfunction	Description
Process sensor workloads	Execute perception and localization computations from the onboard sensing pipeline.
Run navigation and control tasks	Execute planning, decision-making, and low-level control functions within the available processing budget.
Manage concurrent execution	Support simultaneous operation of sensing, planning, communication, and actuation services during the mission.

Table 12: Functional decomposition of the computation platform.

1.3.7 Software Architecture

Subfunction	Description
Manage inter-module communication	Exchange data between perception, localization, navigation, and actuation modules in a structured and timely manner.
Provide autonomy services	Support transforms, navigation services, and integration utilities required by the onboard software stack.
Coordinate subsystem execution	Maintain coherent sequencing between sensing, planning, control, collection, and delivery operations.

Table 13: Functional decomposition of the software-coordination subsystem.

To ensure modularity and real-time reliability on the Raspberry Pi, the software stack will be organized into a layered architecture. Each layer communicates via the ROS2 DDS middleware, decoupling high-level decision-making from low-level hardware constraints. Table 14 showcases an approximate architecture.

Subfunction	ROS2 Node(s)	Description
Mission Control	Brain	Coordination: Implements a State Machine to sequence tasks (Search $\rightarrow$ Pickup $\rightarrow$ Deliver).
Navigation	Nav2 / Planner	Autonomy: Computes global and local paths using costmaps. Handles obstacle avoidance and recovery behaviors (e.g., spinning if stuck).
Perception	Slam_Toolbox	Localization: Processes LIDAR scans and Odometry (IMU, Encoders) to maintain the map and robot pose (x, y, θ) via a TF2 transform tree.
Vision	Depth_Vision	Detection: Processes Depth Camera streams to identify blocks. Publishes relative distances to the /block_pose topic to trigger the collection sequence.
Actuation	Hardware_Bridge	Control: Interfaces with the motor drivers. Translates velocity commands into wheel speeds and manages the Conveyor belt and wheels PWM signals.

Table 14: Layered ROS2 architecture for the software-coordination subsystem.

1.4 Subsystem Design Exploration

Several design spaces include multiple credible solutions with different trade-offs in robustness, implementation complexity, and resource demand. Selected options are marked with green.

1.4.1 Wheel Solutions

Option	Principle	Advantages	Limitations	Relevance to the project
Differential-drive chariot	Two actuated wheels and passive supports provide planar locomotion.	Low mechanical complexity, low cost, and straightforward chassis integration.	Non-holonomic motion and more demanding movement modeling than a holonomic platform.	Preferred baseline in the source document; suitable for the expected arena surfaces when combined with ball casters.
Omni-wheel platform	Four omni-wheels provide holonomic motion.	Lateral maneuverability and easier alignment with targets.	Requires four actuators and a more complex mechanical implementation.	Attractive for precision maneuvers, but less aligned with the simplicity and power objectives of the project.

Table 15: Comparison of movement solutions.



Figure 2: Ball casters used as passive wheels.

1.4.2 Button Interaction Solutions

Option	Principle	Advantages	Limitations	Relevance to the project
Rigid finger	A dedicated rigid appendage presses the door button.	Stable contact geometry and simple fabrication, for example by 3D printing.	Requires accurate localization and alignment with the button.	Preferred because it isolates the task into a simple, dedicated interaction mechanism.
Rear plane of the robot	The chassis body is used as the contact interface.	No additional mechanical part is required.	Requires substantial reorientation near the door and may complicate access to the opening.	Mechanically simple, but operationally less efficient than a dedicated contact element.

Table 16: Comparison of button-interaction solutions.

1.4.3 High-level localization solutions

Option	Principle	Advantages	Limitations	Relevance to the project
LIDAR + SLAM	Use range sensing and simultaneous localization and mapping to infer pose from arena geometry.	High pose accuracy and direct wall-based map generation.	High integration complexity and non-negligible engineering cost.	Highest-ranked localization concept in the original analysis.
Wheel odometry	Integrate motion from wheel encoder measurements.	Simple implementation and no dependence on external references.	Accumulates drift and is sensitive to wheel slip.	Useful as a baseline estimator and as an input to fused localization.
IMU	Use inertial measurements to estimate orientation and motion cues.	Provides heading information without wheel-ground assumptions.	Position estimates drift over time.	Appropriate as a complementary sensor, but insufficient as a standalone localization solution.
Visual markers	Use known visual markers as absolute references.	Provides absolute corrections when markers are visible.	Limited range and sensitivity to lighting conditions.	Potentially useful for relocalization, but not robust enough to be the primary localization method.

Table 17: Comparison of high-level localization solutions.

Several solutions have been selected. Encoder's odometry, IMU and LIDAR data will need to be fused with an state estimator. This will offer redundant sensing and high robustness to individual sensor weaknesses although with the cost of higher implementation complexity.

Option	Principle	Advantages	Limitations	Relevance to the project
Kalman Filter (sensor fusion)	Recursive probabilistic estimator that combines system dynamics with noisy sensor measurements to estimate the robot state.	Provides optimal state estimation for linear Gaussian systems; efficiently fuses encoders, IMU, and range sensors while reducing noise and drift.	Requires model tuning and linear system assumptions; implementation complexity higher than simple odometry.	Selected solution because it enables reliable sensor fusion and improves localization robustness across the arena.
Particle Filter	Monte Carlo method representing the robot state distribution with many sampled hypotheses (particles).	Handles non-linear systems and non-Gaussian noise; robust to ambiguous observations.	Computationally expensive and may require a large number of particles for accurate estimation.	Suitable for complex localization problems but unnecessarily heavy for the computational constraints of the platform.
Moving average / low-pass filtering	Smooths sensor measurements by averaging or applying a low-pass filter to reduce noise.	Very easy to implement and computationally inexpensive.	Does not perform true state estimation and cannot fuse multiple sensor models effectively.	Useful for signal smoothing but insufficient as a primary localization estimator.

Table 18: Comparison of candidate state estimation methods for robot localization. The selected approach is highlighted.

1.4.4 Obstacle avoidance and block detection, collection and delivery solutions

Option	Principle	Advantages	Limitations	Relevance to the project
Depth camera (block detection)	A single sensor provides both visual information and range estimation.	Direct block detection and distance estimation from the same measurement stream.	Higher acquisition cost and a moderate processing load.	Preferred for combined block detection and relative ranging.
Camera + image processing	Monocular imaging combined with onboard vision algorithms.	Flexible visual detection with lower sensor cost.	Sensitive to lighting conditions and more algorithmically demanding.	Viable if cost dominates, but less robust than direct depth sensing.
LIDAR (obstacles)	Laser ranging measures obstacle geometry and can support coarse target detection.	Accurate range, wide field of view, and long sensing distance.	Expensive and mechanically constraining because of sensor placement.	Strong candidate for obstacle detection; block discrimination is less direct than with vision.
Infrared sensors	Short-range infrared proximity sensing.	Precise at short range and inexpensive.	Narrow field of view and very limited range.	More appropriate for local proximity or internal sensing than for global block search.
Ultrasonic sensor	Time-of-flight ranging with acoustic waves.	Low cost and simple electronics.	Limited accuracy, moderate field of view, and short useful range.	Suitable only as a supplemental obstacle sensor.

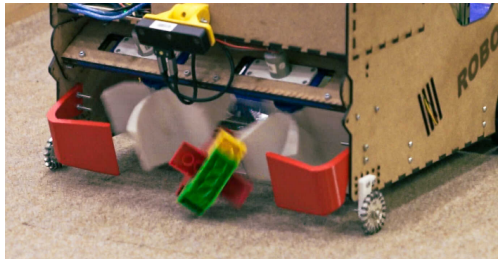
Table 19: Comparison of block- and obstacle-detection sensing solutions.

Option	Principle	Advantages	Limitations	Relevance to the project
Classical Image Analysis with OpenCV	Use classical computer-vision operations to localize blocks in camera images.	Low implementation overhead and lightweight computation.	Sensitive to lighting and environmental variation.	Pragmatic baseline for onboard block detection.
PyTorch-based machine learning or deep learning	Use trained models for block detection and localization.	Can improve robustness and accuracy in difficult edge cases.	Higher compute demand and a non-trivial data and training pipeline.	Relevant only if classical vision does not meet the required detection performance.

Table 20: Comparison of block-detection software approaches.

Option	Principle	Advantages	Limitations	Relevance to the project
Fans	Use compliant blades to pull blocks into the robot.	High expected collection effectiveness and compliance with block shape variability.	Requires two actuators and careful dimensioning of the intake.	Highest-ranked collection concept in the original analysis.
Ramp	Use passive geometry and forward motion to guide blocks upward into storage.	Simple mechanism with low mechanical complexity.	Standalone effectiveness must be validated because blocks may be pushed away instead of captured.	Attractive as a low-complexity option, but performance risk is significant.
Sweepers	Use compliant sweeping elements to redirect blocks inward.	Mechanically compliant and potentially tolerant to impact.	Structural stiffness and collection efficiency are uncertain.	Worth considering as a secondary concept if fan integration becomes impractical.

Table 21: Comparison of block-grabbing solutions.



(a) Fans.



(b) Sweepers.

Figure 3: Block-grabbing solutions.

Option	Principle	Advantages	Limitations	Relevance to the project
Conveyor belt	A motorized conveyor transports stored blocks out of the robot during delivery.	Fast mechanism, high storage compatibility, front-side discharge, and no reset motion after release.	Requires mechanical clearance from the ground, at least one actuator, and supporting bearings.	Preferred because it combines storage and release efficiency.
Drivable back door	An actuated hatch opens to release the payload.	Keeps blocks contained and releases on command.	Requires an additional actuator and hinge mechanism.	Viable, but less favorable than a conveyor if repeated delivery cycles are required.
PVC strip curtain	Passive flexible strips retain blocks until they are pushed out.	Simple and passive containment concept.	Uncertain reliability because the blocks are light and may not generate enough force to exit consistently.	Low-complexity idea, but the release reliability appears insufficient for competition use.

Table 22: Comparison of delivery solutions.



Figure 4: Strip curtain with conveyor belt solution.

1.4.5 Power system solutions

Option	Principle	Advantages	Limitations	Relevance to the project
Battery voltage for motors + one regulated 5 V rail	Motors are supplied directly from the battery, while electronics share a regulated 5 V output.	Simple architecture that satisfies the mandatory 5 V digital requirement.	Restricts the selection of electronics to components compatible with 5 V regulation.	Preferred because it matches the stated requirements with minimal power-system complexity.
Battery voltage for motors + multiple regulated outputs	Motors remain on battery voltage, while electronics use multiple dedicated rails.	Supports a broader range of digital components, including industrial voltage levels.	Increases power-system complexity.	Relevant if sensor or compute choices require voltages beyond 5 V.
Single regulated 5 V output for all components	Both motors and electronics are supplied from a regulated 5 V rail.	Stable motor speed over battery discharge and a uniform power architecture.	Limits motor choice to 5 V devices and places high current demand on the regulator.	Viable only for a very small platform with modest actuation requirements.
Multiple regulated outputs for all components	Every subsystem receives a dedicated regulated voltage rail.	Broadest component compatibility.	Overly complex relative to the current project scale.	Offers flexibility beyond the likely needs of the robot.

Table 23: Comparison of power-architecture options.

1.4.6 On Board Computer solutions

Option	Principle	Advantages	Limitations	Relevance to the project
Raspberry Pi	General-purpose single-board computer running the autonomy stack.	Straightforward ROS integration and a compact all-in-one computing solution.	Available compute headroom may be limited for heavy perception pipelines.	Preferred because it matches the intended software ecosystem and integration effort.
NVIDIA Jetson Nano	Embedded computing platform with higher parallel processing capability.	Higher computing capacity for demanding perception workloads.	More expensive, higher power consumption, and likely beyond project needs. More complex to set up.	Appropriate only if the final perception stack exceeds Raspberry Pi capability.

Table 24: Comparison of onboard computing platforms.

1.4.7 Localization solutions

Option	Principle	Advantages	Limitations	Relevance to the project
SLAM Toolbox	2D LIDAR and odometry-based SLAM for mapping and localization.	Efficient in CPU and memory, and supports dynamic map updates when the environment changes.	Limited to 2D and sensitive to odometry quality.	Best fit among the listed mapping packages for an embedded platform such as a Raspberry Pi.
Cartographer	LIDAR and IMU-based trajectory optimization and mapping.	High precision and reduced accumulation of localization error in larger indoor environments.	More difficult to implement and potentially too demanding for the intended compute platform.	Technically strong, but resource intensive relative to project constraints.
RTAB-Map	Visual and depth-based SLAM with loop closure.	Provides a 3D representation of the environment.	High RAM demand and limited camera field of view reduce practicality on the current platform.	Relevant only if a depth-camera-centric architecture is retained and enough compute is available.

Table 25: Comparison of mapping and localization software options.

In addition to the SLAM toolbox the TF2 transform library would be necessary in a ROS2 based implementation to maintain a consistent spatio-temporal transform tree between robot, sensors, map, and targets. allowing the robot to retain target positions in a common reference frame.

1.4.8 Navigation & Path Planning solutions

Table 26: Comparison of navigation and path-planning options.

Option	Principle	Advantages	Limitations	Relevance to the project
TSP with clustering	Partition the target set into clusters and solve route ordering within each group.	Balances route quality with adaptability to distributed targets.	High implementation complexity.	Best fit for a mix of goal setting and block collection during the traject.
Greedy nearest-target selection	Repeatedly choose the closest available block.	Simple, fast, and easy to implement.	Can be strongly suboptimal in global score or travel distance.	Strong baseline or fallback strategy for early integration.
Reactive forward-search heuristic	Drive until a block is encountered and collect it immediately.	Minimal planning overhead and no initial computation delay.	Slow overall behavior and a high risk of missing profitable targets.	Useful only for early prototyping or degraded operation.
Dijkstra	Compute shortest paths over a weighted graph representation of the arena.	Optimal graph search with a simple conceptual formulation.	Can become slow on large maps.	Good low-risk route planner once a map graph is available.

Option	Principle	Advantages	Limitations	Relevance to the project
A*	Use heuristic graph search to compute shortest paths efficiently.	Optimal with an admissible heuristic and typically faster than Dijkstra.	Performance depends on heuristic quality.	Strong candidate for route execution in a mapped arena.
Hierarchical clustering + TSP	Solve route ordering at multiple scales.	Efficient when many blocks must be serviced.	Very high implementation complexity.	Relevant only if the observed target set becomes large.
Semi-mapping of visible blocks	Plan over the subset of blocks currently observed by the robot.	Avoids purely reactive behavior while limiting the planning problem size.	Difficult to implement robustly under partial observability.	Promising middle ground between greediness and global planning.
Full TSP	Solve the complete tour over all blocks.	Mathematically optimal.	Intractable once the number of targets becomes moderate.	Not suitable for real-time competition execution.
Dynamic programming	Apply exact optimization over small target subsets.	Optimal on small groups.	Memory intensive.	Potentially useful as a local optimizer, not as the primary mission strategy.
Global SLAM-based block inventory	Map every block before starting collection.	Provides precise routes to known targets.	Delays mission start and is computationally expensive.	May improve route quality, but conflicts with fast deployment during a competition run.
Reinforcement learning	Learn a collection policy from simulation or data.	Can be robust to edge cases not captured by hand-tuned logic.	High training cost, high compute demand, and simulation-to-reality risk.	Not compatible with the project's schedule and resource constraints.

1.4.9 Middleware solutions

Option	Principle	Advantages	Limitations	Relevance to the project
ROS 2 with Nav2	Modern robotics middleware combined with a reusable navigation framework.	Robust, modular, and aligned with current industry practice.	Steeper learning curve than simpler custom software.	Best match for integrating perception, transforms, localization, and navigation into one autonomy stack.
ROS	Legacy robotics middleware.	Large user community and a broad tutorial base.	Obsolete relative to ROS 2 and more vulnerable to architectural limitations.	Useful only as a fallback if ROS 2 integration becomes impossible.
Custom DDS publisher/subscriber architecture	Build a project-specific communication layer directly on DDS concepts.	Full control of the software stack.	Not realistic within the available schedule.	Incompatible with the stated timeline constraints despite its conceptual flexibility.

Table 27: Comparison of middleware and software-architecture options.

1.5 Mission Strategy

The selected strategy prioritizes the difficult-access zones early in the run so that the robot can exploit the door and ramp while its storage is empty and its maneuverability is maximal. The sequence currently retained for implementation is the following:

1. Start from the initial, initialize system, reset the localization origin and move toward the Zone 3 access route while collecting accessible blocks in Zone 1. Path would be optimised to collect the blocks that lie on the direct path to the button.
2. Align with the door button, actuate it, and enter Zone 3 before revisiting lower-priority regions of the arena.
3. Collect the available blocks associated with the Zone 3 phase, then return to the emptying zone at the collection corner to unload the first payload. If the block storage has space collect extra blocks in the direct path to origin.
4. Unload the first payload. Use know corner locations and ground truth for localization.
5. Navigate from the collection corner toward the ramp, climb into Zone 4, and collect all blocks assigned to that zone before beginning the final return.
6. Return to the emptying zone, release the remaining payload, and only then consider any residual low-cost opportunities if time remains.

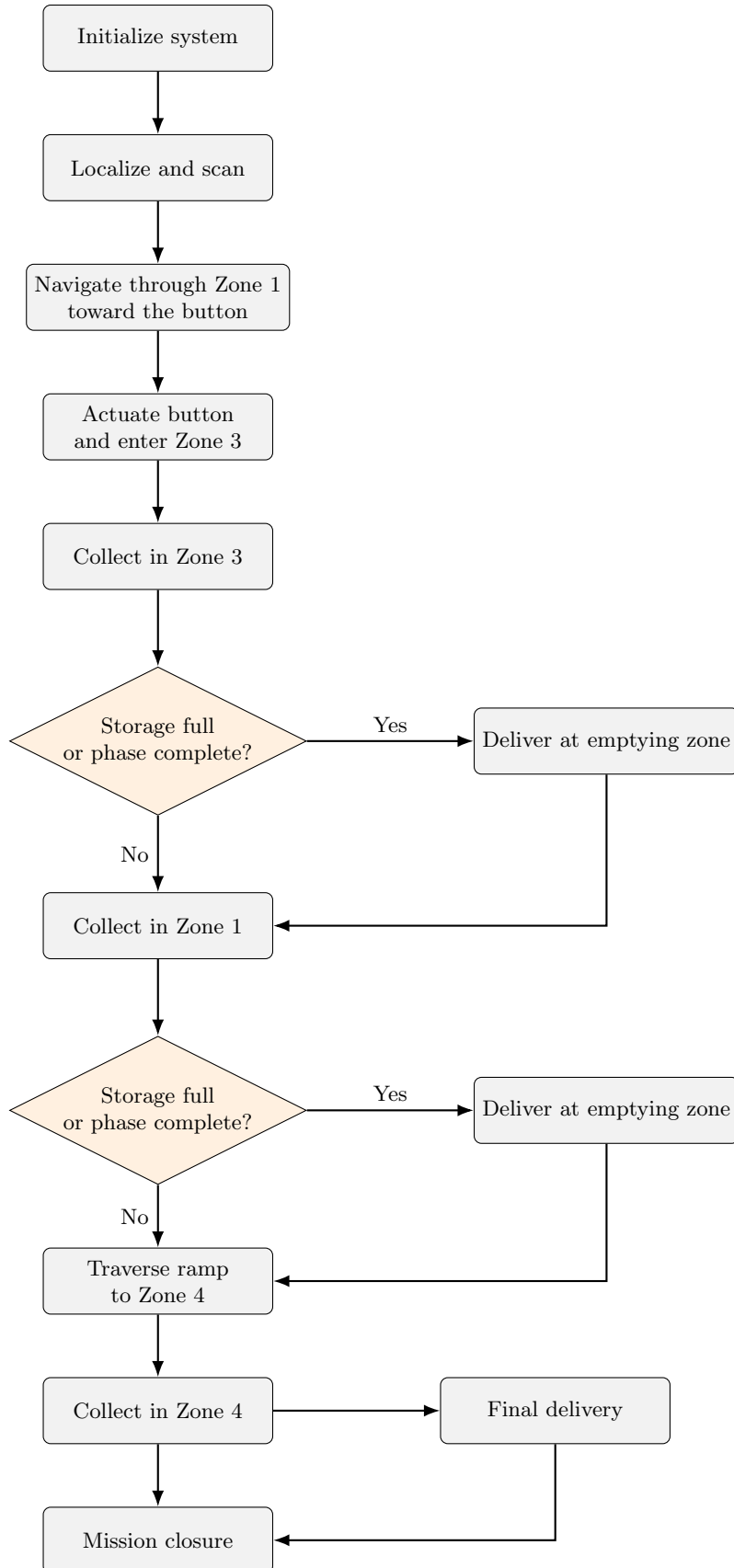
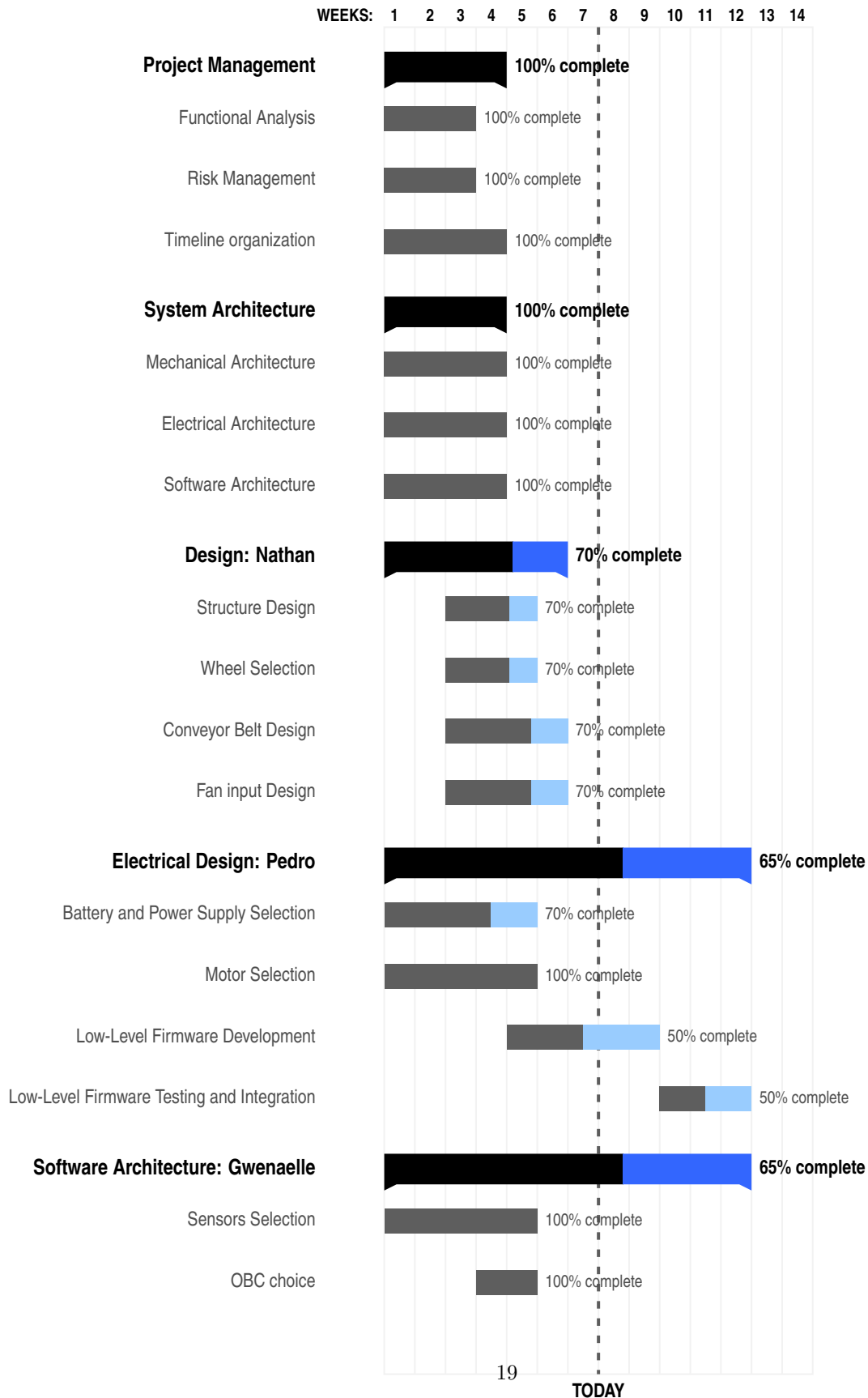
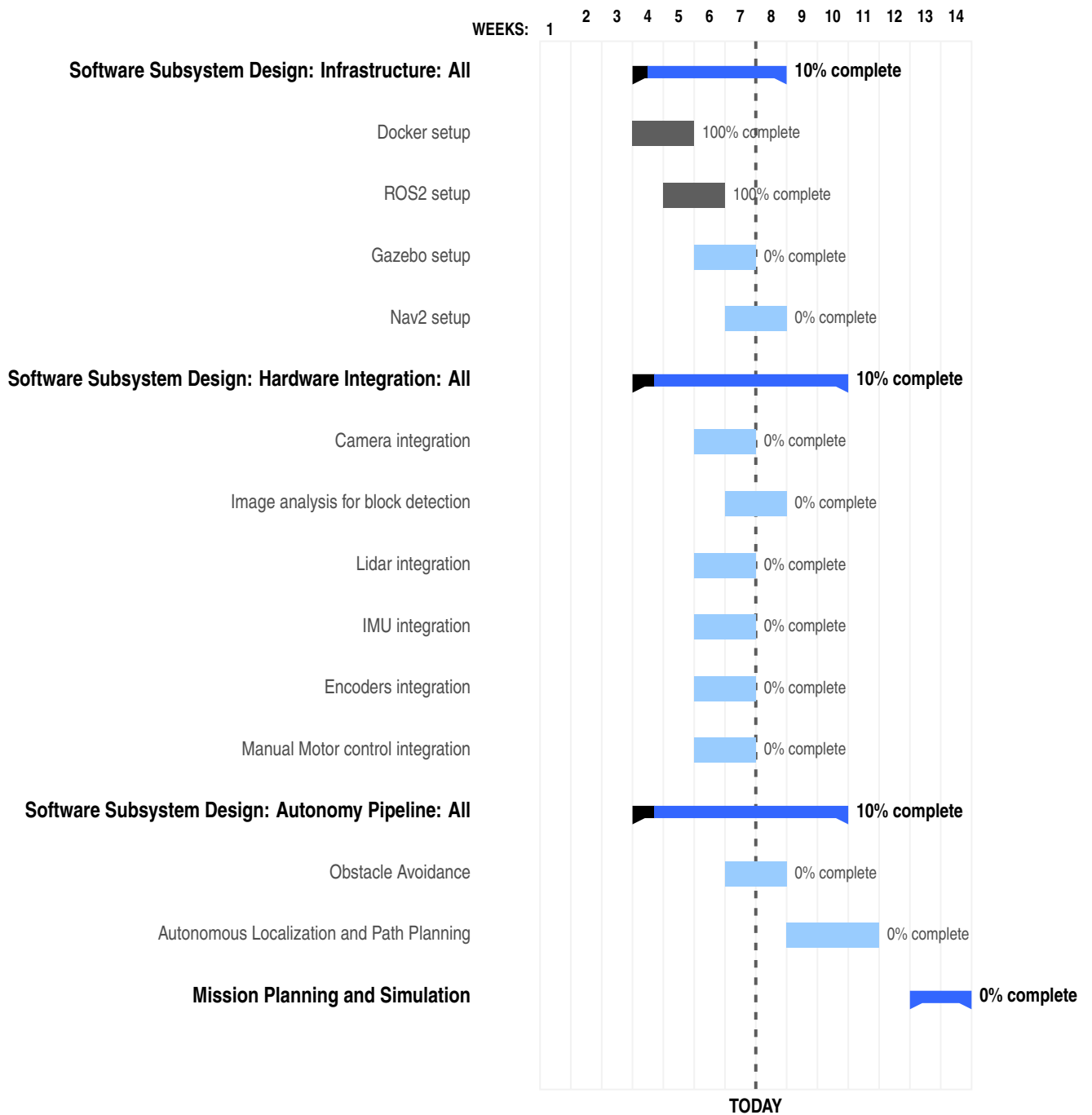


Figure 5: Operational functional flow of the robot during the competition.

2 Project Timeline





2.1 Up to date BOM

Quantity	Component	Price (CHF)
3	DFRobot DRI0018 2x15A motor driver	135.00
5	DFRobot FIT0403 90:1 DC motor with encoder	161.00
1	Jetson Nano	115.00
1	LiDAR Slamtec RPLIDAR A1M8 360° 12 m	100
1	Luxonis OAK-D Lite-FF depth camera	150.00
2	LiPo Battery 3S (11.1 V) 4000 mAh	80.00
1	LiPo protection & power board	25.00
2	WildTumper 120x60 mm wheel	15.00
TOTAL		781

Table 28: Bill of materials and component cost estimation for the robot platform.

